

278 函

See search also provided in Japan.
 before moving to the next level neighbors.

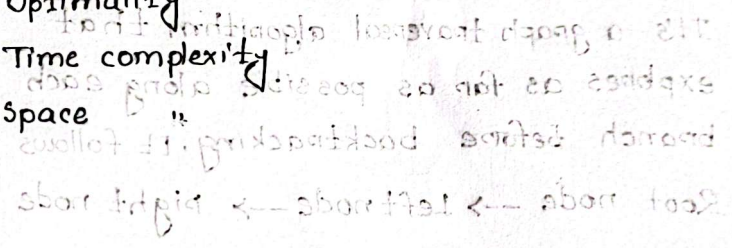
Республика Беларусь

- h.

1940-1941 : 2000-2001
 1941-1942 : 2001-2002

Properties of searching algorithms

- (i) Completeness
- (ii) Optimality
- (iii) Time complexity
- (iv) Space



1000

- V5A

1. Less time
2. Less money

9306 Bay Road - P
 8 Bridgeport Ave
 9306 Bay Road - P
 8 Bridgeport Ave

1. Less time

- Uniform Cost Search
- Depth-Limited Search
- Iterative Deepening Search

BFS

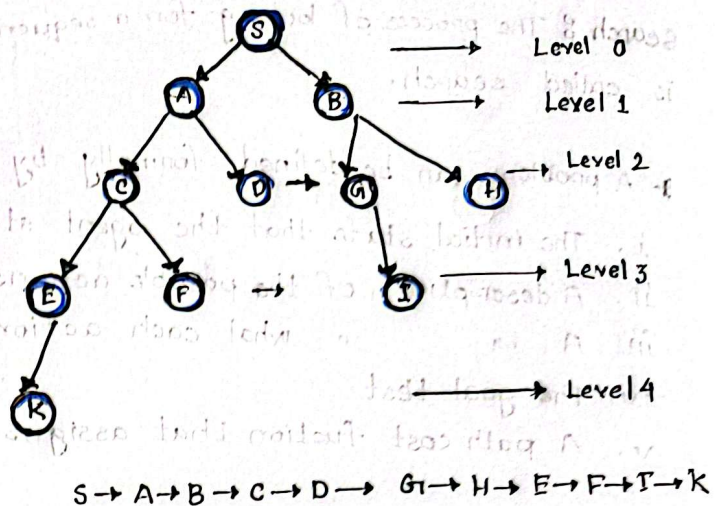
- It visits all the neighbors of a vertex before moving to the next level neighbors.

BFS search algo traverse in layers.

Advantages

- i. Simplest
- ii. Complete.
- iii. If there are multiple solutions, then a minimal soln. will be found.
- iv. Finds the path of minimal length of the goal.

Disadvantages : Lots of memory
Lots of time.



DFS

It's a graph traversal algorithm that explores as far as possible along each branch before backtracking. It follows

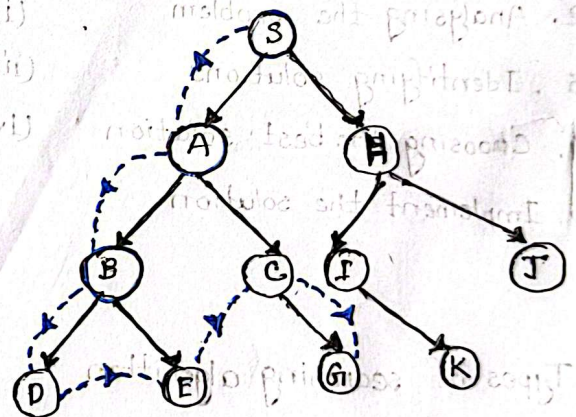
Root node → Left node → Right node

Adv

1. Less memory
2. Less time

Disadv

1. No guarantee of finding solution
2. Goes very deep down searching & sometimes it may go to the infinite loop



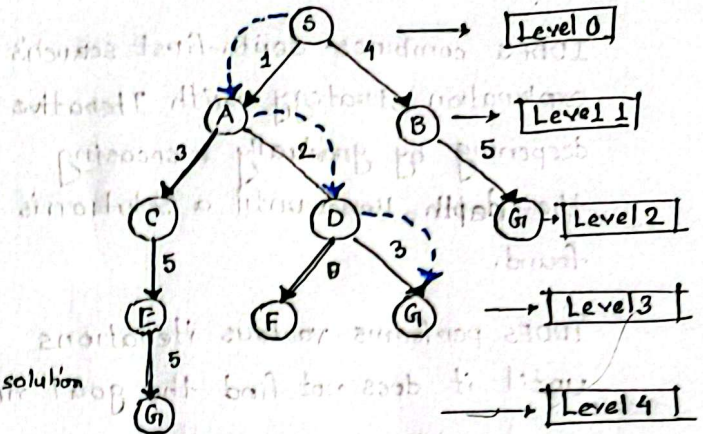
UCS Uniform Cost Search

It explores nodes with the least cost first.

Properties:

1. Completeness : Yes
2. Optimality : Yes
3. Time complexity : $O(b^{1+(C^*/\epsilon)})$
4. Space : $O(b^{1+(C^*/\epsilon)})$

b = branching factor
 C^* = cost of the optimal solution



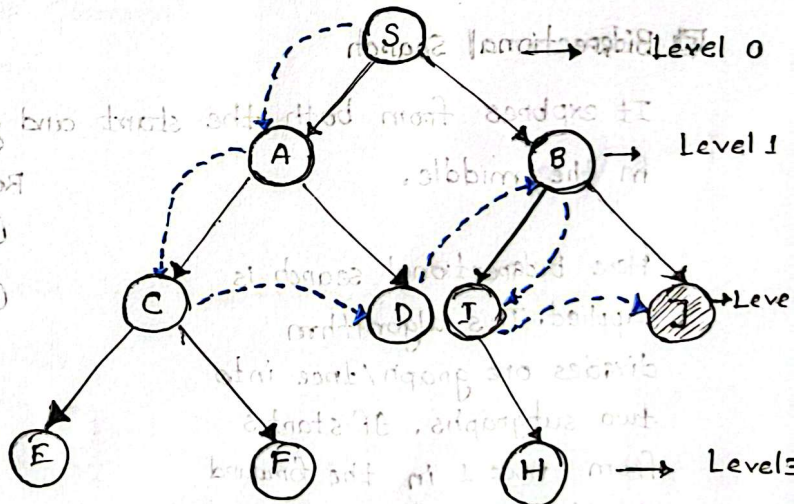
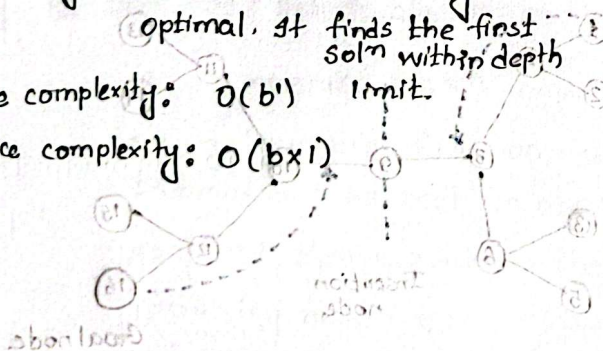
Limitation of DFS

- DFS has a limitation called stack overflow, especially when dealing with deep or infinite graphs, as it uses recursion or an explicit stack to keep track of vertices. To address this, an iterative version of DFS can be implemented using an explicit stack to avoid recursion.

DLS (Depth-limited-search)

DLS is a modification of DFS that limits the depth exploration.

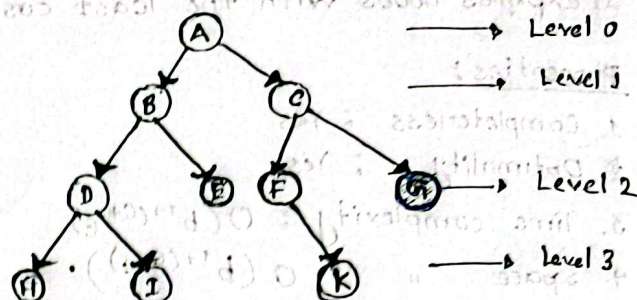
- Completeness : Yes, if the goal is within the depth limit.
- Optimality : DLS is not necessarily optimal. It finds the first soln within depth limit.
- Time complexity : $O(b^l)$
- Space complexity : $O(b \times l)$



IDDFS

IDDFS combines depth-first search's exploration strategy with Iterative deepening by gradually increasing the depth limit until a solution is found.

IDDFS performs various iterations until it does not find the goal node.



1st iteration ---> A

2nd " ---> A, B, C

3rd " ---> A, B, D, E, C, F, G

4th " ---> A, B, D, H, I, E, C, F, K, G

In the 4th iteration, the algorithm will find the goal node.

Completeness : IDDFS is complete

Optimality : IDDFS " optimal

Time Comp : $O(b^d)$

Space " : $O(bd)$

b = branching factor

d = maximum depth reached by any iteration

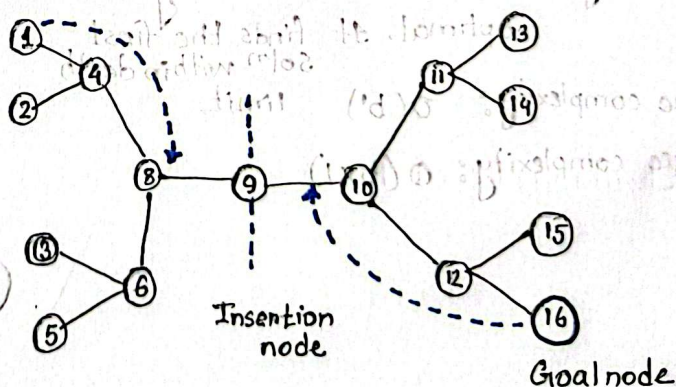
Bidirectional Search

It explores from both the start and goal nodes simultaneously, meeting in the middle.

Here bidirectional search is applied. This algorithm divides one graph/tree into two subgraphs. It starts from node 1 in the forward direction and starts from the goal node 16 in the backward direction.

The algo terminates at node 9 where two searches meet.

Root node



Completeness : Yes, if we use BFS in both search
 Optimal : Yes
 Time Comp : $O(bd)$
 Space Comp : $O(bd)$

- **Optimal solution** : In one whose measure of quality is close to the best that could theoretically be obtained.
- **Frontier** : a set of leaf nodes
- **Loopy path** : infinite, unsolvable
- **Explored set** : keeps track of the nodes that have already been expanded.

☐ Informed Search (Heuristic search)

- uses problem specific knowledge to find solutions efficiently, unlike uninformed search algorithms.

- Searching strategy : define by picking of the order of node expansion.
- Heuristic function $h(n)$: estimated cost of the cheapest path from node n to a goal node.

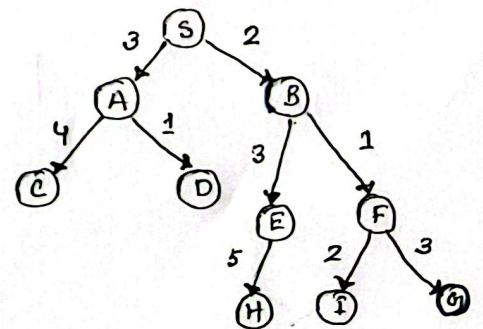
1. Best first search algo (Greedy)
2. A* Search Algorithm.

☐ Best first search algorithm

is an informed search algorithm that selects nodes to explore based on a heuristic function, which estimates the cost to reach the goal. It always explores the most promising node first according to the heuristic evaluation.

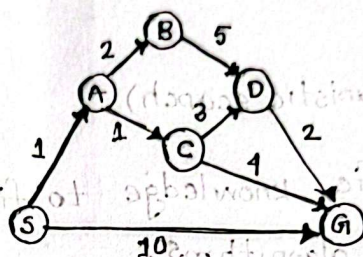
Time : $O(b^m)$
 Space : $O(b^m)$

Completeness : incomplete
 Optimality : is not optimal



Minimizing the total estimated cost solution. A^* is similar to UCS except that it uses $g(n) + h(n)$ instead of $g(n)$

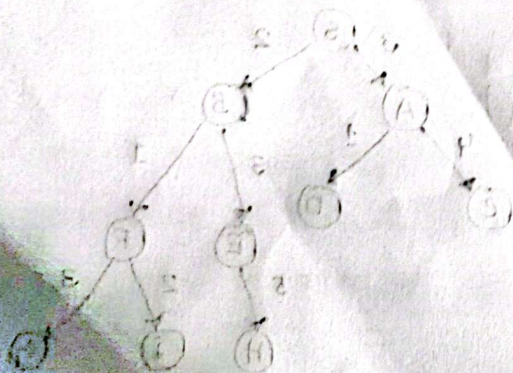
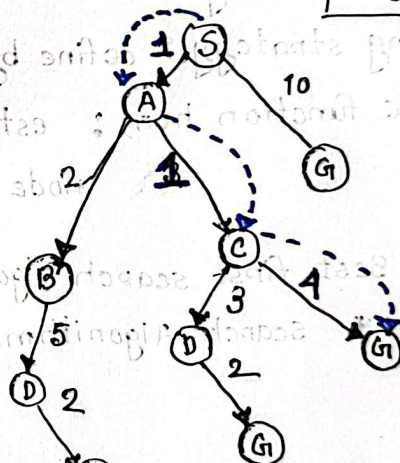
$$f(n) = g(n) + h(n)$$



state	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Optimality of A*

$h(n) > \text{actual cost}$

$$h(n) = \text{actual cost}$$
$$h(n) < \text{actual cost}$$


the heuristic evaluation, the heuristic explores the most promising nodes first, according to a heuristic function, which estimates the cost to reach the goal. It always explores the most promising nodes first, according to a heuristic evaluation.

Optimality : is not optimal
Completeness : incomplete
Space : $O(b^m)$
Time : $O(b^m)$